EE6483 Artificial Intelligence and Data Mining
Mini Project (Option 1): Sentiments of Product Reviews

Team 110: MENG XIANGYU, WU DAYU, YIN JIYANG

November 2026

Group members and contributions

Name	Matric no.	Email	Contribution
MENG XIANGYU	G2607496E	`meng0106@e.ntu.edu.sg`	Shared code (`common.py`, data split, evaluation), TF-IDF baselines, noisy-label experiment, resource/accuracy analysis; Sections 2.1, 3, 8 and 10; report integration.
WU DAYU	G2603624F	`dayu002@e.ntu.edu.sg`	GloVe features, averaged-GloVe + LR, BiLSTM model and tuning; Sections 2.2, 2.4, 5 and 9.
YIN JIYANG	G2605135H	`jiyang002@e.ntu.edu.sg`	RoBERTa fine-tuning, ensemble and final submission, error analysis; Sections 2.3, 4, 6 and 7; README.

All members contributed papers to the literature table in Section 2.2 and cross-reviewed each other's sections.

Abstract

We study document-level sentiment classification of product reviews (7,401 labeled training reviews, 85.4% positive; 1,851 test reviews). Using one grouped, stratified validation split that prevents duplicate-text leakage, we compare three feature formats: sparse TF-IDF n-grams with linear classifiers, static GloVe embeddings with logistic regression and a BiLSTM, and subword tokens with a fine-tuned RoBERTa-base model. The average of three RoBERTa runs reaches 0.962 accuracy and 0.927 Macro-F1 on validation data, against 0.930 and 0.857 for the best TF-IDF model, while the BiLSTM only matches TF-IDF at about 80 times its training time. We also discuss adaptation to unlabeled hotel reviews and, in a controlled experiment with injected label noise, show that Confident-Learning-style filtering is the most reliable of three remedies (ROC-AUC 0.909 versus 0.837 without a remedy at 20% symmetric noise).

1 Introduction

Sentiment analysis determines the polarity of a piece of text, and product reviews are one of its most common applications. This report builds and evaluates classifiers that label a product review as negative (0) or positive (1). The training set contains 7,401 labeled reviews, of which 85.4% are positive, and the test set contains 1,851 unlabeled reviews. The task combines a modest amount of training data, a strong class imbalance and noisy, informal text, so the choice of feature format matters for both accuracy and cost.

We make four contributions. (1) We define one grouped, stratified validation split in which duplicate review texts never cross the split, and use it for every model, so that all scores are comparable. (2) We compare three feature formats of increasing cost under identical conditions: sparse TF-IDF n-grams with linear classifiers, static GloVe embeddings with logistic regression and a BiLSTM, and subword tokens with a fine-tuned RoBERTa-base model. (3) We submit the average of three RoBERTa runs, which reaches 0.962 accuracy and 0.927 Macro-F1 on the validation split, and we analyze its predictions on a manually labeled sample of test reviews. (4) We discuss how the approach transfers to unlabeled hotel reviews and, with a controlled label-flipping experiment, how noisy ratings degrade it and which of three remedies helps most.

Section 2 reviews the literature. Sections 3–5 describe the data, the models and the parameter settings; Sections 6–8 report the test-set results, the analysis of predictions and the comparison of feature formats;

Sections 9 and 10 address the two transfer scenarios, and Section 11 concludes. Table 1 shows where each question of the assignment is answered, and Appendix A explains how to reproduce the results.

Table 1: Where each question of the assignment is answered.

Question	Topic	Section
(a)	Literature survey	2 (2.1–2.4)
(b)	Feature format and data preprocessing	3
(c)	Model, architecture, loss and training	4
(d)	How parameters were determined	5
(e)	Test-set results and `submission.csv`	6
(f)	Correctly and incorrectly classified samples	7
(g)	Feature format, resources and accuracy	8
(h)	Hotel reviews without rating scores	9
(i)	Hotel reviews with noisy rating scores	10

2 Related Work

2.1 Problem definition, challenges and solution types

Problem definition. Sentiment analysis, also called opinion mining, is the computational study of the opinions, sentiments and emotions that people express in text[1]. A common formalization describes an opinion as a quintuple (e, a, s, h, t): an entity e, one of its aspects a, the sentiment s towards that aspect, the opinion holder h and the time t. This project is the simplest instance of that definition: *document-level binary polarity classification*. The entity is the purchased product and the holder is the reviewer, both implicit, and each review receives one label for the whole document. Given a review x, we learn a classifier $f(x) \approx P(y{=}1 \mid x)$, where $y = 1$ is positive and $y = 0$ is negative, and predict $\hat{y} = \mathbb{1}[f(x) \geq \tau]$ for a decision threshold τ. Neutral or irrelevant reviews are not modeled.

Granularity. Sentiment can be analyzed at three levels of granularity:

- *Document level*: one label per review, as in this project.
- *Sentence level*: usually a subjectivity filter followed by polarity classification.
- *Aspect level*: aspect-based sentiment analysis (ABSA) extracts aspects such as "fit" or "fabric" and assigns a polarity to each[2].

Related tasks use finer label sets, such as the five-point scale of the Stanford Sentiment Treebank, or emotion categories.

Challenges. The main difficulties, each illustrated with what we observe in our data:

1. **Negation and contrast.** Words such as "not", "never" and "but" change the polarity of the words around them. In "Not bad for just a shirt" (from our test set), "bad" signals a negative review, yet the review is mildly positive. A bag-of-words model loses the word order needed to resolve this.
2. **Implicit sentiment, sarcasm and irony.** Many reviews contain no explicit opinion word ("the seam came apart after one wash"), or say the opposite of what they mean.
3. **Mixed polarity.** A review may praise one aspect and criticize another. The document label then depends on which aspect the reviewer weighs more.
4. **Domain dependence.** The polarity of a word depends on the domain. "Small" is negative for a hotel room and often positive for a phone, and "unpredictable" is positive for a film plot but negative for a car's steering. Classifiers trained on one product category therefore degrade on another[3].
5. **Class imbalance.** Product reviews are mostly positive: 85.4% of our training reviews are positive and 14.6% negative. Predicting "positive" for every review already gives 85.4% accuracy, so we report Macro-F1 and the F1 of the negative class alongside accuracy.
6. **Noisy and informal text.** Reviews contain typos, slang, emoticons and very different lengths. Our training reviews have a median of 42 words, but the longest has 1,066. When star ratings are used as labels, some labels disagree with the text, which is the problem studied in Section 10.

Solution types under different settings. Table 2 organizes common approaches along the three axes named in the assignment; Wankhade et al.[1] give a broader recent overview.

Supervised versus unsupervised. Supervised methods learn from labeled reviews. They range from Naive Bayes, maximum-entropy and SVM classifiers on bag-of-words features, and their strong NB-SVM variant, through convolutional and recurrent networks over word embeddings, to fine-tuning pre-trained Transformers such as BERT[4] and its successors. Unsupervised, lexicon-based methods need no labels:

- scoring phrases by their association with seed words such as "excellent" and "poor";
- looking words up in a sentiment lexicon such as SentiWordNet;
- rule-based scoring with VADER.

They transfer easily across domains but are usually less accurate than supervised models. Between the two extremes lie weakly supervised methods. Distant supervision uses emoticons or star ratings as noisy labels, and semi-supervised methods use self-training on unlabeled reviews. Zero-shot and few-shot prompting of large language models is competitive on simple binary polarity, but still falls behind fine-tuned models on fine-grained and structured sentiment tasks[5].

Closed-set versus open-set. In the closed-set setting every test review belongs to one of the training classes, here positive or negative. In the open-set setting, test inputs may come from classes never seen in training: neutral reviews, spam, text that is not a review, or another language. A closed-set classifier still forces each of these into "positive" or "negative". Open-set methods add a rejection option. DOC replaces the softmax with one-vs-rest sigmoids and rejects an input when no class is confident enough. Confidence-based detection flags inputs whose maximum softmax probability is low.

Without versus with domain shift. When training and test data come from the same distribution, standard supervised learning applies. Under domain shift, labeled data from a source domain (e.g. books) must serve an unlabeled target domain (e.g. kitchen appliances). Approaches include:

- structural correspondence learning, which aligns domain-specific words through shared "pivot" features[3];
- stacked denoising autoencoders that learn domain-invariant representations;
- domain-adversarial training (DANN);
- continued pre-training of the language model on target-domain text.

These methods are the basis of our answer to Section 9.

Table 2: Solution types for sentiment classification under different settings. Our project is marked with ⋆.

Setting	Typical methods	Strengths / limitations
Supervised ⋆	BoW + NB/LR/SVM; CNN/RNN over embeddings; fine-tuned Transformers[4]	Most accurate when labeled in-domain data exist; needs labels and is domain-specific.
Unsupervised	Seed-word association; lexicons; rules (VADER)	No labels needed and interpretable; misses context, domain-specific terms and sarcasm.
Weak / semi-supervised	Distant labels from emoticons or ratings; self-training; LLM prompting[5]	Cheap labels at scale; labels are noisy (see Section 10).
Closed-set ⋆	Softmax over a fixed label set	Simple; inputs from unseen classes are forced into a known class.
Open-set	One-vs-rest rejection (DOC); confidence thresholds	Can reject neutral or irrelevant inputs; needs extra thresholds.
No domain shift ⋆	Standard train/validation/test protocol	Validation accuracy predicts test accuracy.
Domain shift	SCL pivots[3]; denoising autoencoders; DANN; domain-adaptive pre-training	Uses unlabeled target text; gains depend on how far apart the domains are.

Our setting. The project is *supervised, closed-set and in-domain*. Training and test reviews come from the same product-review distribution; 101 of the 1,851 test reviews even appear verbatim in the training set. The data are imbalanced, and the labels may contain some noise. We therefore build supervised classifiers. A TF-IDF linear model serves as the baseline, and sequence models and pre-trained language models serve

as improvements; Section 2.4 motivates this choice. We use threshold tuning and Macro-F1 to handle the imbalance.

2.2 Recent progress

Table 3 covers domain adaptation, sentiment-aware representation learning, and evaluation under distribution shift. The published results use different benchmarks and protocols.

Table 3: Selected recent work on sentiment classification and transfer.

Year	Method	Data	Contribution	Reported result
2021	UDALM [6]	Amazon Reviews, 12 transfers	Joint source classification and target masked language modeling	91.74% accuracy, 1.11 percentage points above the prior comparison.
2022	Memory-based contrastive learning [7]	Two Amazon review datasets and a rumor dataset	Queue-based supervised contrastive learning for unseen domains	Improved over the tested domain adaptation methods.
2022	Mere contrastive learning [8]	Amazon Reviews and FDU-MTL	In-batch negatives separate classes in representation space	90.39% mean accuracy over 12 Amazon transfers.
2022	Multi-domain TSA [9]	Yelp for self-training, three target evaluation datasets	Augments training with weakly labeled examples across domains	Showed gains across target datasets.
2024	SentiEval [5]	26 datasets over 13 sentiment tasks	Compares prompted LLMs and task-trained smaller models	Simple classification was more favorable to LLMs than fine-grained structured tasks.
2024	SoftMCL [10]	Sentence- and aspect-level sentiment and other affective tasks	Uses soft valence and a momentum queue during pretraining	Showed benefits across several sentiment tasks.
2025	Synthetic instructions [11]	12 sentiment benchmarks including Amazon, Yelp and IMDb	Builds instruction examples without labeled review instances	Reported a 5.1-point gain over more complex instruction tuning methods.
2025	BESSTIE [12]	Google Place and Reddit texts in three English varieties	Tests sentiment and sarcasm across varieties	Performance fell on Indian English, especially for sarcasm.

Domain adaptation studies suggest using hotel text during training even without ratings. Cross-variety evaluation and the LLM comparisons also motivate keeping a held-out target sample when testing transfer. For the present product-review task, we compare sparse lexical and dense embedding models on the same split before drawing conclusions about transfer.

2.3 Leading research groups

Erik Cambria's research group at Nanyang Technological University studies how commonsense knowledge can help a system judge the sentiment of a text and explain its decision. SenticNet 7 and SenticNet 8 show how this line of work developed. The first paper focuses on building a knowledge base that links different expressions to shared concepts. The second extends this approach by examining what those concepts mean in a sentence.

SenticNet 7 (2022). Cambria et al. [13] use XLNet to propose words and phrases that could express a similar meaning in context. A similarity measure based on links to emotion concepts in ConceptNet helps separate expressions that may look alike in context but carry different feelings. The resulting knowledge base includes both single words and phrases. For example, it treats "dead right" as a phrase with its own meaning, so the word "dead" does not determine its sentiment on its own.

The authors compared SenticNet 7 with 20 sentiment lexicons on ten datasets, including Amazon reviews and IMDb reviews. They attributed part of its performance to wider vocabulary coverage, since missing entries were a common source of errors for other lexicons. Sarcasm and sentences that express opposing opinions about different targets still caused errors. A larger vocabulary helps the system recognize more expressions, while deciding which opinion applies to which target remains a separate problem.

SenticNet 8 (2024). SenticNet 8 combines a commonsense knowledge base with a hierarchical attention network that identifies which tokens are most relevant to the sentiment decision [14]. Its text processing goes

from standardizing the wording to grouping expressions with similar meanings, then to describing the situation those expressions refer to. The paper illustrates this with several ways of describing a phone purchase. Phrases such as "buy a phone" and "pay for a phone" can both be represented as a change in who owns the phone. This example makes the purpose of the intermediate representation easier to see.

On the Sentiment140 polarity task, SenticNet 8 achieved 88.80% accuracy, compared with 85.07% for the RoBERTa baseline in the same paper. The test texts were tweets, so applying the method to product reviews would require a separate evaluation.

Relevance to product reviews. Review 1123 in Section 7 illustrates the issue of opinion targets. The reviewer describes old plastic luggage tags cracking and breaking, then explains why the new leather tag was a better purchase. The initial RoBERTa ensemble predicted a negative label, whereas the final human label was positive. An analysis that links the damage to the old tags and the recommendation to the new tag could preserve the distinction that matters for this review. Whether SenticNet would classify this example correctly remains an open question for a direct comparison on the same data.

2.4 Baseline choice and motivation

Word-level TF-IDF with logistic regression is a suitable baseline. It fits sparse review vectors efficiently, and coefficients show which features drive a prediction. Bigrams can capture local phrases such as *not good*. Longer contexts, including a later clause that reverses an earlier opinion, remain difficult with local features. Exact term matching also gives limited coverage to unseen words.

We evaluate two extensions using the same grouped split. First, averaging static word vectors such as Word2Vec or GloVe[15;16] gives a dense 300-dimensional representation with one vector per review. The logistic classifier then tests whether this representation preserves useful sentiment information. Second, a BiLSTM retains token order and learns forward and backward representations before pooling. The Transformer line evaluates contextual subword representations. Because the feature representation and classifier can both change between lines, the score differences describe the performance of each complete system.

At threshold 0.5, the validation Macro-F1 scores are 0.8451 for TF-IDF + LR, 0.7623 for mean GloVe + LR, and 0.8524 for the three-seed BiLSTM. The sequence model improves on averaging in this experiment. Its score is 0.0072 higher than TF-IDF + LR on this fold. The comparison uses a single validation split.

3 Data and Feature Representation

3.1 Feature formats and pipeline

We compare three feature formats of increasing cost, one per model line, so that Section 8 can measure the trade-off directly:

1. **Sparse bag-of-n-grams with TF-IDF weighting**, used by linear classifiers;
2. **static pre-trained word embeddings** (300-dimensional GloVe[16]), used by an averaged-embedding classifier and by a BiLSTM;
3. **subword tokens fed to a pre-trained Transformer** (RoBERTa-base), whose contextual representation is fine-tuned end-to-end. This is the format of our final model.

Figure 1 summarizes the pipeline. All three formats share the loading, auditing, splitting and cleaning steps described below.

3.2 Data audit

Both files are read as JSON tables. The training set has 7,401 reviews with a binary label (6,319 positive, 1,082 negative). The test set has 1,851 unlabeled reviews. No review is missing or empty. Review length varies widely: the median is 42 words, 75% of reviews have at most 70 words, 97% at most 178, and the longest has 1,066 words. We also checked for duplicates, comparing texts after lower-casing and collapsing whitespace:

- 154 distinct texts occur more than once in the training set, covering 373 rows, and every copy of a text has the same label;
- 101 test reviews are identical to a training review;

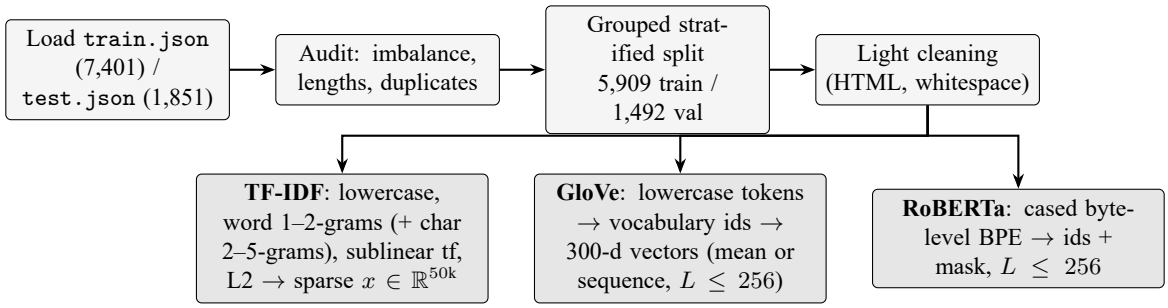


Figure 1: Data preprocessing pipeline. Every vectorizer, vocabulary and model is fitted on the training part only; the validation split is used only for model selection and the test set only for the final predictions.

– 21 test reviews duplicate another test review.

3.3 Grouped, stratified validation split

Every model is tuned on one shared validation split. If duplicates were split at random, copies of the same review would land on both sides, and the validation score would partly measure memorization. We therefore group rows by their normalized text and take fold 0 of a 5-fold `StratifiedGroupKFold` (seed 42). This yields 5,909 training and 1,492 validation reviews. The validation set contains 218 negatives (14.6%, the same ratio as the full set), and no text appears on both sides. Cross-validation inside the training part, including the internal folds used to calibrate the SVM, uses the same grouping. The test set is used only to produce `submission.csv`.

3.4 Text cleaning

We unescape HTML entities, collapse runs of whitespace and strip leading and trailing spaces. We deliberately do *not*:

– remove stop words, because standard stop lists contain “not”, “no” and “but”, which carry sentiment;
– stem, or strip punctuation such as “!” and “?”.

Text is lower-cased for TF-IDF and GloVe, and kept in its original case for RoBERTa, whose tokenizer is case-sensitive.

3.5 TF-IDF features (L1)

The text is tokenized with a pattern that keeps single-character tokens (e.g. the “t” of “don’t”) as well as “!” and “?”. We extract unigrams and bigrams; bigrams capture short negations such as “not good” and “not worth”. Each term t in review d is weighted with sublinear term frequency and smoothed inverse document frequency,

$$w_{t,d} = \left(1 + \log \mathrm{tf}_{t,d}\right) \cdot \left(\log \frac{1+N}{1+\mathrm{df}_t} + 1\right), \tag{1}$$

where N is the number of training reviews and df_t the number of reviews containing t. Each review vector is then L2-normalized. The vectorizer is fitted on the training rows only. Three settings were tuned by grouped 5-fold cross-validation:

– the minimum document frequency (1, 2 or 3);
– the vocabulary cap (10k, 30k, 50k or unlimited);
– whether to append character 2–5-grams, which can absorb misspellings.

The selected representation has 50,000 word 1–2-gram features. It is very sparse: 86 non-zeros per review on average, a density of 0.17%.

3.6 Static word embeddings (L2)

The L2 tokenizer lowercases and unescapes the text, retains contractions and the punctuation tokens `!` and `?`, and keeps up to 256 tokens. The vocabulary is built only on the training portion, with minimum frequency two and special `<pad>` and `<unk>` entries. Of 6,796 ordinary training vocabulary entries, 6,359 have a vector in the 300-dimensional `glove.6B` table[16]. On validation, 3.17% of token occurrences map to `<unk>`, and 4.95% have no usable pretrained vector after applying the vocabulary. Mean GloVe averages only the covered

vectors. BiLSTM gives each in-vocabulary entry missing a GloVe vector and the shared unknown entry a random initial vector. Padding is excluded from both directions and the pooled statistics.

3.7 Subword tokens for a pre-trained Transformer (L3)

After cleaning, reviews retain their original casing. RoBERTa's byte-level BPE tokenizer converts the text to token IDs and adds the `<s>` and `</s>` tokens. The 256-token limit includes these special tokens. When a review is too long, the tokenizer keeps the beginning of the text. Each batch is padded to the length of its longest sequence, and the attention mask distinguishes input tokens from padding. Section 4 describes how the model uses the first token's representation for classification.

3.8 Labels and class imbalance

Labels are used as given (0 = negative, 1 = positive). Each model outputs $P(y{=}1 \mid x)$. We report metrics at threshold 0.5 and at thresholds tuned on the validation split: one maximizing accuracy (τ_{acc}, used for the submission) and one maximizing Macro-F1 (τ_{F1}). The RoBERTa runs used unweighted cross-entropy. Section 5 describes the parameter settings evaluated for the other models.

4 Models

4.1 TF-IDF classifiers

The three TF-IDF classifiers take the sparse n-gram vectors described in Section 3. Logistic regression maps a weighted sum of the features to a positive-class probability. The linear SVM produces a margin score, which is converted to a probability by sigmoid calibration on grouped folds within the training split. Complement Naive Bayes estimates each class's feature weights from reviews in the other class, with additive smoothing.

The feature and classifier settings were selected by grouped five-fold cross-validation within the training portion. Each selected model was then fitted on 5,909 reviews and evaluated on the shared validation set of 1,492 reviews. For test prediction, both the TF-IDF vectorizer and the classifier were refitted on all 7,401 labeled reviews.

4.2 GloVe embeddings and BiLSTM

The mean-embedding classifier averages the GloVe vectors available for the tokens in a review. If none of the tokens has a pre-trained vector, the review is represented by a zero vector. The resulting 300 features feed a regularized logistic regression classifier.

The BiLSTM uses the same training vocabulary to initialize an embedding matrix of size $V \times 300$, where V is the vocabulary size. Entries without pre-trained vectors are initialized from a zero-mean Gaussian with standard deviation 0.05, and the padding row is fixed at zero. The input is a batch of integer token IDs of shape $B \times L$, where B is the batch size and $L \leq 256$.

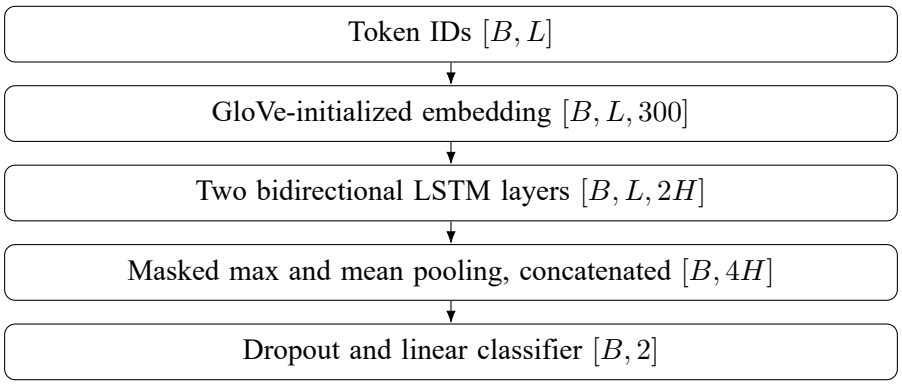


Figure 2: GloVe-initialized BiLSTM. H denotes the hidden width per direction.

In each layer, the backward direction reverses only the valid part of the sequence. Both pooling operations ignore padding. Max pooling keeps the strongest activation in each channel, while mean pooling summarizes the valid states. The two pooled vectors are concatenated to give the classifier $4H$ features, or 512 when $H = 128$.

The BiLSTM was trained with Adam and cross-entropy. In the weighted variant, class j receives a loss weight of $n/(2n_j)$, where n is the number of training reviews and n_j is the number in that class. The best epoch was selected by validation Macro-F1 at a threshold of 0.5. Softmax gives the positive-class probability, and the validation and test probabilities were averaged across seeds 42, 43, and 44.

4.3 RoBERTa-base

Input and classifier. After the light cleaning described in Section 3, each review retains its original casing. The RoBERTa tokenizer splits the text into subwords and limits the sequence to 256 tokens, including the special tokens. Longer sequences are truncated at the end. Within each batch, shorter sequences are padded to match the longest one, and an attention mask distinguishes the input tokens from padding.

The encoder produces a 768-dimensional representation at each token position. The classification head reads the representation at the first token, `<s>`. A dense layer with `tanh` activation transforms this vector, with dropout before and after the layer during training. The final linear layer gives one score per class. Softmax converts the two scores to probabilities, and the positive-class probability is retained for prediction.

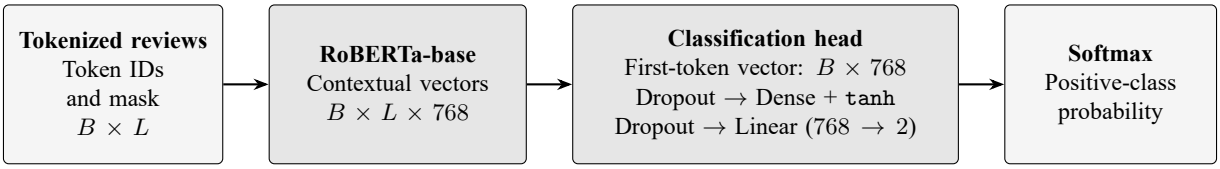


Figure 3: RoBERTa-base architecture for review classification. B is the batch size and $L \leq 256$ is the sequence length after padding within the batch. The head uses the encoder output at the first token, `<s>`.

Fine-tuning. The encoder and classification head were fine-tuned together on the 5,909 training reviews from the shared split. Each of the three final runs used one epoch, FP32 precision, a batch size of 4, and unweighted cross-entropy. AdamW used a weight decay of 0.01. The learning rate increased to 2×10^{-5} over the first 10% of training steps and then decayed linearly to zero. Section 5 describes how we selected this schedule.

Prediction. The data split and training settings were fixed across the three runs, with random seeds 42, 43, and 44. For each review, the three positive-class probabilities were averaged before applying the decision threshold. Section 6 explains how the threshold was chosen on the validation set and reports the resulting test predictions.

5 Parameter Selection

Sparse baseline. The sparse models use grouped five-fold cross-validation within the 5,909 training reviews. The TF-IDF grid covers minimum document frequency, a vocabulary cap and optional character features. These control the trade-off between coverage of rare expressions and the number of features. Logistic regression uses $C = 10$ and balanced class weights with 50,000 word unigram and bigram features. The calibrated linear SVM selects $C = 1$ without class weights, and ComplementNB selects $\alpha = 0.1$ with 10,000 features.

Static embedding models. For mean GloVe, we compare $C \in \{0.1, 1, 10\}$ with and without balanced class weights. At the common threshold of 0.5, the $C = 10$ balanced setting has the highest Macro-F1 (0.7623). Its unweighted counterpart scores 0.7598 Macro-F1 and 0.8988 accuracy, versus 0.8539 accuracy with weighting. The higher negative F1 with weighting (0.6148 versus 0.5770) explains the selection despite the accuracy trade-off. The BiLSTM begins with Adam, learning rate 10^{-3}, hidden width 128 per direction, dropout 0.3, trainable embeddings, and ordinary cross-entropy. A sequential search changes learning rate (10^{-3} or 5×10^{-4}), hidden width (64, 128 or 256), dropout (0.3 or 0.5), embedding freezing, then class weighting. This limits cost and isolates each comparison near the incumbent configuration. A factorial search would be needed to estimate interactions between settings. The learning rate controls step size, dropout reduces overfitting, and freezing tests whether source supervision benefits from updating GloVe. Class weighting changes gradients during training, while threshold tuning changes the operating point after training. Each run uses batches of 64, gradient clipping at 1.0, up to 10 epochs and patience of two epochs.

Candidates are ranked by validation Macro-F1 at threshold 0.5. The selected BiLSTM configuration is repeated with seeds 42, 43 and 44. We use the standard deviation to describe run-to-run variability. Since the same validation fold also guides tuning and early stopping, these are model-selection scores. The unlabeled product test set is used for prediction. For each model, the accuracy and Macro-F1 thresholds are then fitted separately on the validation probabilities.

The sequential sweep ran seven configurations, followed by the three final fits. The selected learning rate is 0.001, with width 128 per direction, dropout 0.3, trainable embeddings, and ordinary cross-entropy. The single-seed Macro-F1 is 0.8383 ± 0.0050 (mean and population standard deviation). The three-seed probability average reaches 0.8524 Macro-F1 at threshold 0.5. Its validation-selected thresholds are $\tau_{\text{acc}} = 0.500$ and $\tau_{\text{F1}} = 0.500$. The individual settings and scores are recorded in the project repository (Appendix A).

Transformer. We initially fine-tuned RoBERTa-base for one epoch with AdamW, a constant learning rate of 2×10^{-5}, and weight decay of 0.01. We used unweighted cross-entropy, a batch size of 4, a maximum sequence length of 256 tokens, and FP32 precision. At a threshold of 0.5, seeds 42 and 43 reached validation Macro-F1 scores of 0.897 and 0.884, respectively. Seed 44 scored 0.461 and classified every validation review as positive.

Repeating seed 44 under the original settings reproduced the same Macro-F1 score. Extending training to two epochs at the same constant learning rate also left Macro-F1 at 0.461 at the end of the second epoch. This variation across seeds suggested that the original fine-tuning setup was sensitive to randomness. The seed affects both classification-head initialization and the order of training examples, so our runs could not isolate their individual effects.

We returned to one epoch and changed the learning-rate schedule. The learning rate rose to 2×10^{-5} over the first 10% of training steps and then decayed linearly to zero. Seed 44 improved to 0.914 Macro-F1 at the same threshold. We then reran seeds 42 and 43 under the same schedule so that all three runs used the same settings. The final Macro-F1 scores were 0.920, 0.910, and 0.914 for seeds 42, 43, and 44, respectively. In the following sections, the *initial ensemble* is the average of the three constant-learning-rate runs and the *final ensemble* is the average of the three warmup runs.

6 Test-Set Results

Final model. We averaged the positive-class probabilities from the three RoBERTa-base runs trained with 10% warmup and linear decay. At a threshold of 0.5, the population standard deviation of single-run Macro-F1 decreased from 0.203 under the original schedule to 0.004 across the same three seeds. On the validation set, the final RoBERTa ensemble achieved 0.927 Macro-F1 after threshold selection. The 75% RoBERTa and 25% TF-IDF + SVM blend achieved 0.926 after its own threshold selection, so we used the RoBERTa ensemble for submission.

Decision threshold. On the 1,492 labeled validation reviews, a threshold of 0.590 gave the highest accuracy (0.962) and Macro-F1 (0.927) among the thresholds evaluated for the RoBERTa ensemble. We applied the same threshold to the test set, assigning label 1 to reviews with an average positive-class probability of at least 0.590 and label 0 to the remaining reviews.

Test predictions. Table 4 shows the predicted label distribution for the 1,851 unlabeled test reviews.

Table 4: Predicted label distribution on the 1,851 test reviews.

Predicted label	Number of reviews	Share
Positive (1)	1,568	84.71%
Negative (0)	283	15.29%
Total	1,851	100.00%

The file `submission.csv` contains the `reviews` and `predictions` columns in the original test-set order.

7 Analysis of Predictions

Test sample and annotation. The error analysis in this section uses the initial RoBERTa ensemble, trained with a constant learning rate and evaluated at a threshold of 0.680. After excluding 101 test reviews whose

text also appeared in the training set, we selected 40 reviews for manual annotation. Fifteen were closest to the initial RoBERTa decision threshold, 15 were drawn from reviews on which that ensemble and TF-IDF + SVM disagreed, and 10 were drawn at random from the remaining reviews. Two members labeled all 40 independently and agreed on 38 (95%). A third member resolved the two disagreements.

Correct and incorrect predictions. RoBERTa matched the final human labels on 31 of the 40 sampled reviews (0.775). The sample deliberately included reviews near the decision threshold and reviews on which the models disagreed, so this accuracy applies only to the selected reviews. On the same 40 reviews, the final RoBERTa ensemble described in Section 6 matched 34 human labels (0.850).

RoBERTa correctly classified reviews 274 and 99. In review 274, the reviewer points out a limitation of the garment and still recommends it for smoothing under clothes. That recommendation supports the positive label. In review 99, the reviewer initially liked the bra's shape and support, but the underwire eventually broke through. This failure supports the negative label despite the initial praise.

RoBERTa classified review 549 as positive even though the reviewer reported a broken boot strap and asked about returning the boots. It classified review 1123 as negative. In that review, the cracking and breaking described by the reviewer concerned the old plastic luggage tags. The reviewer was satisfied with the new leather tag, which was the product being evaluated.

Validation-set errors. On the shared validation set of 1,492 reviews, TF-IDF + SVM, BiLSTM and RoBERTa misclassified 109, 104 and 69 reviews, respectively. Of the 30 reviews that all three models misclassified, 12 contained mixed sentiment. RoBERTa correctly classified 66 reviews that SVM missed, including nine cases categorized as involving context or negation. For example, in validation review 523, the phrase "not disappointed" expresses approval, and the reviewer ends with a recommendation.

8 Feature Format, Accuracy and Resources

The feature format decides two things: what information reaches the classifier (word identity only, word similarity, or word meaning in context), and how much computation and memory are needed to obtain it. We first compare the three formats in general terms (Table 5), then report what we measured on this dataset (Table 6), and finally isolate the effect of feature dimensionality inside the TF-IDF line (Table 7).

Dimensionality within one format. Table 7 shows clearly diminishing returns. Growing the TF-IDF vocabulary from 1k to 30k n-grams raises Macro-F1 from 0.811 to 0.850 and AUC from 0.930 to 0.951. Beyond 30k the gain stops: keeping all 123k n-grams changes AUC by less than 0.001. Because the representation is sparse, the cost grows slowly with the nominal dimension. Each review activates only 45–98 entries, and the training matrix stays under 7 MB. Appending character 2–5-grams is the exception. It multiplies the non-zeros per review by about 7 (649 vs. 98), the matrix size by 6.5, and the training time by 5. The validation Macro-F1 rises by about 0.01, but grouped 5-fold cross-validation shows no gain (0.8514 with character n-grams vs. 0.8515 without). A difference of 0.01 Macro-F1 is within the noise of a split with only 218 negatives, so the cheaper word-only representation was selected.

Across formats: accuracy. The three formats differ in how much linguistic structure they expose. TF-IDF already reaches 0.928–0.930 accuracy, because most reviews contain strong, explicit opinion words. The largest LR weights are intuitive: "disappointed", "uncomfortable", "returned" and "poor" for negative reviews, "great", "comfortable", "perfect" and "love" for positive ones. However, the unigram "not" alone receives the most negative weight. As a result, "not bad" gets only a small positive bigram weight (+0.59), which barely offsets the strongly negative "not" and "bad", while "not worth" (-3.31) is captured correctly. This is exactly where order-aware and contextual features should help: negation, contrast ("but the…" is among the top negative features) and implicit sentiment. The measured results in Table 6 confirm this. At the validation-selected threshold of 0.590, the RoBERTa ensemble reached 0.962 accuracy and 0.927 Macro-F1, compared with 0.930 accuracy and 0.857 Macro-F1 for TF-IDF + SVM at its respective tuned thresholds. ROC-AUC was 0.981 for RoBERTa and 0.950 for the SVM. The three individual RoBERTa runs had a population standard deviation of 0.004 in Macro-F1 at a threshold of 0.5. The BiLSTM, by contrast, only matches TF-IDF: its three-seed average reaches 0.852 (single runs 0.838 ± 0.005), against 0.851–0.857. With 5.9k training reviews, static embeddings and a recurrent encoder learned from scratch capture no more than

Table 5: Comparison of the three feature formats used in this project.

	TF-IDF n-grams (L1)	Static GloVe embeddings (L2)	Contextual subwords, RoBERTa (L3)
Representation	Sparse, one dimension per n-gram (here 50k; 0.17% non-zero)	Dense, 300 per token; mean-pooled or sequential	Dense, 768 per subword token, context-dependent
Word order / context	Only local, via bigrams	Mean: none. BiLSTM: learned sequentially	Full self-attention over 256 tokens
Similar words	Unrelated dimensions ("great" vs "excellent")	Close in vector space	Close, and disambiguated by context
Out-of-vocabulary words	Dropped (2.1% of validation tokens, 24% of word types)	`<unk>` or random vector (3.2% of validation tokens are `<unk>`; 5.0% have no pre-trained vector)	None: unknown words are split into subwords
Trainable parameters	$\approx$ dimension $+ 1$ (50k)	Mean+LR: 301. BiLSTM: 2.9M including the embedding table	$\approx$125M
Hardware / training cost	CPU, seconds	CPU, minutes (BiLSTM)	GPU, minutes to hours
Memory	Sparse matrix of a few MB	Embedding table $V \times 300$ (the full GloVe file is $\approx$1 GB)	$\approx$0.5 GB of weights plus activations
Interpretability	High: one weight per n-gram	Low	Low (needs attribution methods)
Data needed	Moderate; benefits from more data	Less, thanks to pre-trained vectors	Least for a given accuracy; knowledge transferred from pre-training

Table 6: Measured accuracy and resource use on the shared validation split (1,492 reviews, 218 negative). Training time is per run (per seed for neural models) and excludes loading data and pre-trained weights. TF-IDF runs are deterministic. Neural models average the probabilities of seeds 42/43/44. The RoBERTa row uses the three runs trained with 10% warmup and linear decay. Its training time is the mean across the three runs, and its peak GPU memory is the maximum across them. Hardware differs by line: TF-IDF on an AMD laptop CPU (16 threads), GloVe models on a 9-thread x86-64 CPU, RoBERTa on an NVIDIA RTX 3070 Ti Laptop GPU.

Model	Feature dim	Params	Acc@0.5	Acc@τ_{acc}	Macro-F1@τ_{f1}	Train (s)	Peak mem (MiB)
TF-IDF + LR	50,000	50,001	0.924	0.928	0.851	1.5	122
TF-IDF + SVM	50,000	150,009[†]	0.929	0.930	0.857	2.8	150
TF-IDF + ComplementNB	10,000	20,000	0.910	0.926	0.857	1.2	155
Mean GloVe + LR	300	301	0.854	0.899	0.788	0.06	351
GloVe + BiLSTM	300/token	2,876,010	0.930	0.930	0.852	121.6	1,112
RoBERTa-base	768/token	124,647,170	0.959	0.962	0.927	113.1	2,434[§]

[†]Three calibrated SVMs, one per internal calibration fold, each with its own weight vector. [§]Peak GPU memory; the other rows report CPU process memory.

Table 7: Effect of TF-IDF dimensionality, with the classifier fixed (LR, C=10, balanced class weights), trained on the training part and evaluated on the validation split.

Features	Dim	Non-zeros / review	Matrix (MB)	Vectorize + fit (s)	Macro-F1@0.5	AUC
Word 1–2-grams, top 1k	1,000	45	3.1	1.2	0.811	0.930
Word 1–2-grams, top 5k	5,000	63	4.3	1.2	0.840	0.946
Word 1–2-grams, top 10k	10,000	70	4.8	1.1	0.845	0.949
Word 1–2-grams, top 30k	30,000	81	5.5	1.3	0.850	0.951
Word 1–2-grams, top 50k	50,000	86	5.8	1.3	0.845	0.950
Word 1–2-grams, all	123,515	98	6.7	1.2	0.846	0.951
+ char 2–5-grams (all)	174,735	649	43.9	6.3	0.858	0.952

the n-gram weights do. Averaging GloVe vectors discards both word order and the identity of rare but decisive words, and drops to 0.788. RoBERTa achieved the highest validation Macro-F1 among the models compared. Section 7 examines examples of errors made by the initial constant-learning-rate RoBERTa model and the TF-IDF baseline.

Across formats: resources. Resource use grows much faster than accuracy. TF-IDF + LR trains in about 1.5 s on a CPU with about 120 MB peak memory, and predicts the 1,492 validation reviews in a few milliseconds. The GloVe line needs the pre-trained embedding file. The full `glove.6B.300d` table has 400k×300 floats, and only the rows of our vocabulary are kept. The BiLSTM fits take 121.6 s per seed on average (four PyTorch CPU threads), about 80 times TF-IDF + LR, excluding the hyper-parameter sweep. Peak memory for the two GloVe models is process RSS. The GloVe and TF-IDF timings come from different CPUs. RoBERTa-base has 124.6M parameters. One fine-tuning epoch took 113.1 s per seed on average on the RTX 3070 Ti Laptop GPU, with a maximum of 2,434 MiB of allocated GPU memory across the three runs. The CPU-trained BiLSTM averaged 121.6 s per seed. These timings reflect the hardware used for each model. Figure 4 plots validation Macro-F1 against training time on a logarithmic scale.

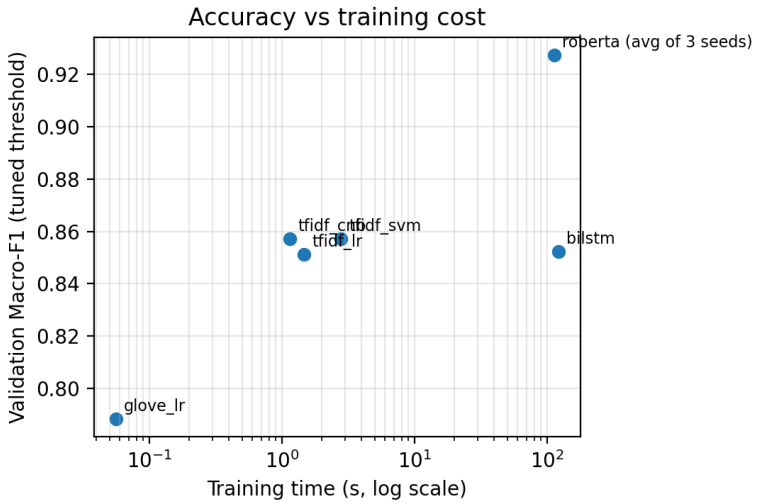


Figure 4: Validation Macro-F1 at the tuned threshold versus training time on a logarithmic scale. For the neural ensembles, training time is reported per seed. The RoBERTa point uses the three warmup runs.

Practical implications.

- **Small labeled data favors pre-trained features.** With only 5.9k training reviews, TF-IDF must learn every n-gram's polarity from scratch, and 24% of the word types in the validation reviews never occur in training. Pre-trained embeddings, and above all pre-trained language models, bring knowledge learned from billions of words, so they generalize better to rare words, typos and paraphrases.
- **Large data or tight budgets favor sparse features.** TF-IDF scales linearly, runs on any CPU and is fully interpretable. It is the right choice when latency, cost or explainability matter more than the last percentage point, and it is a strong baseline.
- **Robustness to domain change differs.** TF-IDF and GloVe fix their vocabulary at training time. A subword model covers any input string, which matters for the hotel-review scenario in Section 9.

In short, for this task the feature format moves accuracy by a few points but cost by orders of magnitude. The choice should therefore follow the accuracy target and the available hardware.

9 Domain Shift: Hotel Reviews Without Ratings

Hotel reviews share general cues such as *excellent* and *disappointing* with product reviews, so a frozen source classifier provides a useful starting point. The meaning of some features can change: *small* can describe a convenient electronic device or an uncomfortably small room. Hotel reviews also discuss service, cleanliness and location, often with conflicting opinions in the same review. Changes in the distribution of topics or class prevalence can make the source-domain decision threshold poorly calibrated on hotel reviews. Prior work

on review transfer provides evidence for these risks, but the magnitude of the loss on hotel reviews requires target-domain evaluation[3;6].

Adaptation with raw hotel text. One option is to continue masked-language-model training on unlabeled hotel reviews and then train the sentiment head on labeled product reviews. Another is to train the sentiment head on source labels while using the hotel corpus for a joint masked-language-model objective, as in UDALM[6]. A cheaper alternative is self-training: label an unlabeled hotel pool with the source model, retain only high-confidence examples from each predicted class, and retrain with the original labeled reviews and a lower weight on pseudo-labels. Confidence cutoffs should be conservative because domain shift can produce confident errors and reinforce class imbalance. For a lightweight linear model, structural correspondence learning can instead use frequent shared pivot words to connect product and hotel vocabulary[3].

Evaluation. If manual annotation is feasible, independently label a random sample of approximately 100 hotel reviews using a fixed overall-polarity rubric. Adjudicate disagreements and document how mixed or neutral reviews are treated. Keep this set separate from the unlabeled adaptation pool. Compare the frozen source model and adapted models on the same labeled reviews using accuracy, Macro-F1 and negative-class F1. Any threshold chosen using hotel labels needs a separate calibration set or a nested validation protocol. Without target labels, inspect OOV rates and prediction distributions as diagnostics and leave target-domain accuracy unreported.

10 Noisy Labels: Hotel Reviews With Inaccurate Ratings

The assignment refers to "the same sentiment classification problem in (g)". We read this as the hotel-review project of Section efsec:hotel: the reviews now come with rating scores, but some of those scores are wrong. Training labels derived from the ratings are therefore *noisy*.

10.1 How our classifier would be affected

Sources and types of noise. Rating errors on review sites are rarely uniform[17]. Three kinds can be distinguished:

- *Random errors*, such as mis-clicks or swapped scales, are roughly *symmetric*.
- *Systematic errors* are *class-conditional*. A guest may write a positive text but give a low score because of the price or a single incident, or a hotel may receive inflated scores. Noise then runs mainly in one direction.
- *Instance-dependent errors* concentrate on ambiguous, mixed or sarcastic reviews, which are exactly the hard cases.

Effect on training. A discriminative classifier fits the labels it is given. Over-parameterized neural networks can memorize even completely random labels. They do tend to learn the clean, dominant patterns first and only later fit the noise. So a fine-tuned RoBERTa or a BiLSTM trained to convergence on noisy ratings would gradually absorb the errors, and its decision boundary and confidence estimates would suffer. Early stopping on clean data delays this but does not prevent it. A regularized linear TF-IDF model cannot memorize as freely, but its weights are biased in the same way. Class imbalance amplifies the damage. With an 85:15 class ratio, flipping each label independently with probability 20% means only 41% of the reviews labeled "negative" are truly negative, because flipped positives outnumber the real negatives. The minority class, usually the one that matters most to a hotel, is affected first.

Effect on evaluation. If the validation and test labels are noisy too, measured accuracy underestimates the true accuracy, and model selection and threshold tuning chase the noise. A small, manually verified clean set is therefore essential for any of the remedies below.

10.2 Experiment: injecting label noise into our data

To quantify these effects we corrupted the labels of our 5,909 training-part reviews and kept the 1,492-review validation split clean. We used two noise models:

- *symmetric*: every label is flipped with probability 0, 5, 10 or 20%;

– *positive→negative*: only positive labels are flipped, with probability 5, 10 or 20%, mimicking a positive text given a low score.

The base model is TF-IDF (word 1–2-grams) + LR, and each setting is repeated with 3 seeds. Methods M1–M3 rely on 5-fold out-of-fold probabilities $p(x)$ computed on the noisy training part, with the same grouped folds as in Section 3. We compared four strategies:

– **M0, baseline**: train on the noisy labels.
– **M1, sample selection (Confident Learning)**[18]. For each class j, set the threshold t_j to the mean predicted probability of j over samples labeled j. A sample is flagged when its most probable class among those with $p_k \geq t_k$ differs from its given label. Flagged samples are removed and the model is retrained.
– **M2, label correction**: a label is flipped when the out-of-fold model assigns more than 0.9 probability to the opposite class.
– **M3, re-weighting**: each sample is weighted by the out-of-fold probability of its given label.

Table 8: Validation results under injected label noise (mean ± std over 3 seeds). Left block: Macro-F1 at threshold 0.5. Right block: ROC-AUC, which does not depend on the threshold. "Neg. prec." is the fraction of training reviews labeled negative that are truly negative. Without class weights and threshold tuning, the clean baseline's Macro-F1 (0.789) is lower than the tuned TF-IDF model in Section 8 (0.851).

		Neg.	Macro-F1 @ 0.5				ROC-AUC			
Noise	Rate	prec.	M0	M1	M2	M3	M0	M1	M2	M3
–	0%	1.00	.789	**.810**	.765	.692	**.950**	.948	.946	.945
Symmetric	5%	0.77	.789	**.814**	.789	.698	.938	**.944**	**.944**	**.944**
	10%	0.60	.772	**.805**	.765	.690	.910	.936	.929	**.939**
	20%	0.41	.720	**.792**	.754	.724	.837	**.909**	.878	.907
Pos→neg	5%	0.77	.802	**.823**	.812	.728	.935	**.944**	**.944**	**.944**
	10%	0.63	.806	**.819**	.809	.769	.917	.940	.936	**.941**
	20%	0.46	.778	**.819**	.796	.814	.901	.931	.917	**.933**

Standard deviations are 0.003–0.020 for Macro-F1 and below 0.008 for AUC; full values are in the project repository (Appendix A).

Findings. Table 8 and Figure 5 lead to five observations.

1. **Noise hurts, and imbalance makes it worse.** Under 20% symmetric noise the baseline's AUC drops from 0.950 to 0.837, and its Macro-F1 from 0.789 to 0.720. The F1 of the negative class falls from 0.63 to 0.52.
2. **Threshold-0.5 metrics can mislead.** Positive→negative noise of 5–10% slightly *raises* the baseline's Macro-F1 at 0.5, although its AUC falls. The extra negative labels merely shift the decision boundary towards the minority class. This is why we also report AUC, and why thresholds must be tuned on clean data.
3. **Sample selection (M1) is the most reliable method.** It gives the best Macro-F1 in every setting, and at 20% symmetric noise it recovers AUC from 0.837 to 0.909. It removed 1,633 reviews on average; 44% of them had actually been flipped, and it caught 61% of all flipped labels. Even on the clean data it removed about 80 hard or ambiguous reviews and improved Macro-F1 by 0.02.
4. **Label correction (M2) is precise but conservative.** 83–97% of the labels it changed were truly wrong, but it changed only 200–280 of them, so its gains are moderate.
5. **Re-weighting (M3) ranks well but needs re-calibration.** It matches M1 in AUC and is best under strong positive→negative noise. However, its Macro-F1 at 0.5 is poor at low noise: negative reviews receive lower out-of-fold confidence, so re-weighting shrinks the minority class further. Combined with class weights or threshold tuning, this weakness would disappear.

These results come from synthetic noise, a linear model and a single validation split. They show the size of the problem and the relative behavior of the strategy families, not the gains to expect on real hotel ratings.

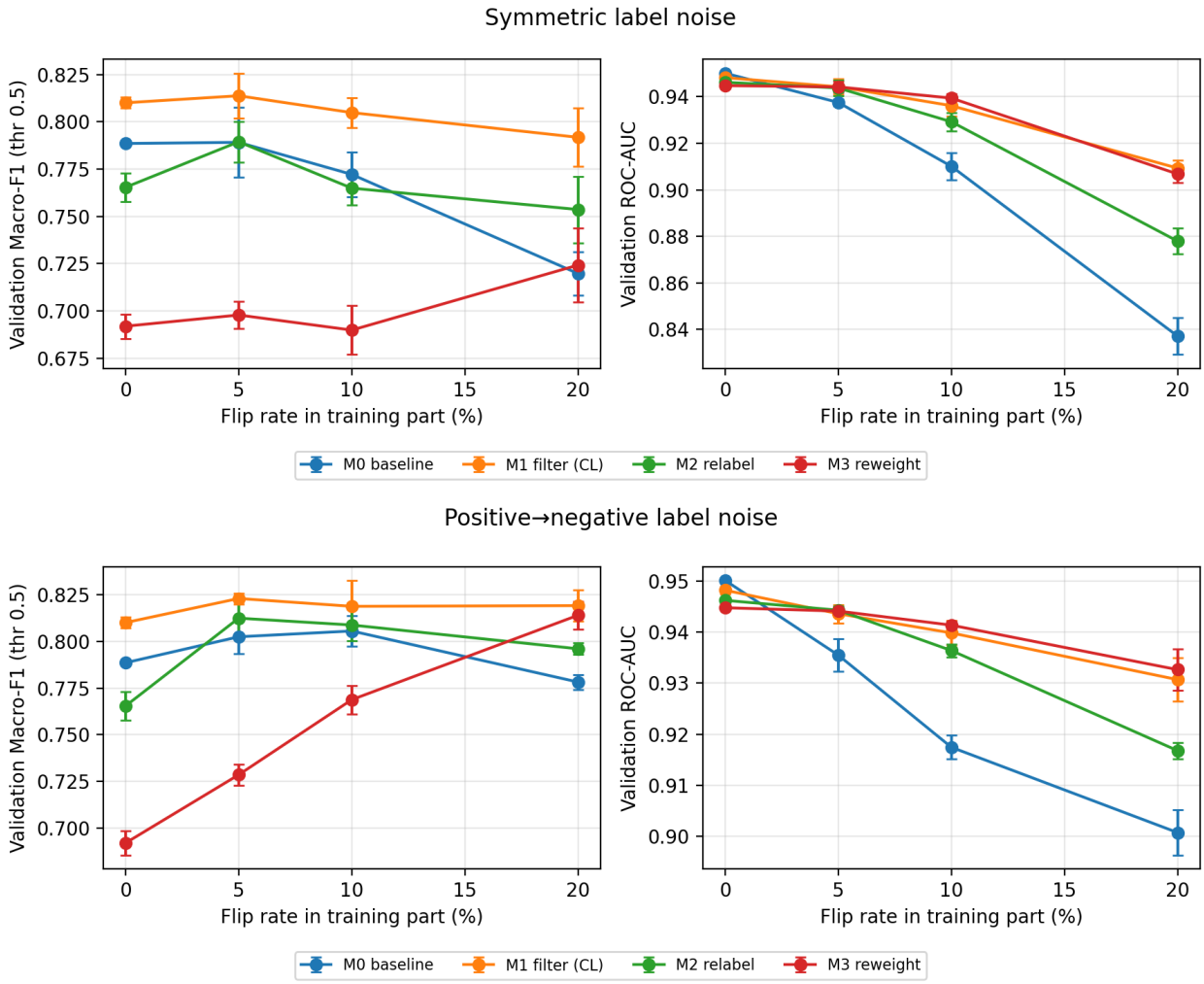


Figure 5: Validation Macro-F1 at threshold 0.5 (left) and ROC-AUC (right) as the training-label flip rate grows. Error bars show ± 1 std over 3 seeds.

10.3 Three approaches to improve the algorithm

The three families below follow the taxonomy of the recent survey by Song et al.[17], where the individual methods named here are reviewed in detail.

1. **Noise-robust losses and loss correction.** Replace plain cross-entropy with a loss that penalizes confidently wrong labels less. Options include generalized cross-entropy, which interpolates between cross-entropy and the noise-tolerant MAE; symmetric cross-entropy; and label smoothing. When the noise is class-conditional, as with positive texts given low scores, one can instead estimate the 2×2 noise-transition matrix T and correct the loss through it (forward/backward correction). T can be estimated from a small clean subset. These methods change only the loss function of our RoBERTa fine-tuning, and add no extra cost.
2. **Sample selection and data cleaning.** Exploit the fact that networks fit clean samples first, so small-loss samples are more likely to be correctly labeled. Co-teaching trains two networks and lets each one update on the small-loss samples chosen by the other. Confident Learning estimates the joint distribution of noisy and true labels from out-of-fold probabilities, then prunes or ranks the suspicious samples[18]; this is our M1, the best method in our experiment. Flagged reviews can also be sent to annotators instead of being discarded.
3. **Label correction and semi-supervised learning.** Instead of discarding suspicious samples, correct or down-weight them. Our M2 and M3 are simple versions. DivideMix fits a two-component mixture model to the per-sample losses to separate likely-clean from likely-noisy reviews. It then treats the noisy ones as *unlabeled* data and trains on both parts with MixMatch-style semi-supervised learning. For hotel reviews, the text itself is a second, independent signal. A zero-shot model, for example our

RoBERTa classifier or an LLM, can check each rating against its text, and the two can be combined into a corrected label.

Recommended pipeline for the hotel project.

1. Manually verify a few hundred reviews to obtain a clean development set, and use it to estimate the noise rate and the transition matrix T.
2. Fine-tune a pre-trained Transformer with a robust loss (GCE or label smoothing) and early stopping on the clean set.
3. Run Confident Learning on out-of-fold predictions; relabel or drop the flagged reviews, sending the uncertain ones to annotators.
4. Retrain, optionally with DivideMix-style semi-supervised learning on the doubtful part, and tune the decision threshold on the clean set.

11 Conclusion

We compared sparse TF-IDF features, static GloVe embeddings and fine-tuned RoBERTa subword representations on a single, leakage-free validation split. A three-seed RoBERTa ensemble performed best, with 0.962 accuracy and 0.927 Macro-F1 at a threshold tuned on validation data, clearly ahead of TF-IDF with a linear SVM (0.930 and 0.857). The BiLSTM only matched TF-IDF while needing about 80 times more training time, and averaged GloVe vectors were the weakest (0.788 Macro-F1). For this task the feature format therefore changed accuracy by a few points but cost by orders of magnitude. Fine-tuning was also sensitive to the random seed: without learning-rate warmup one of three runs collapsed to a constant prediction, and warmup removed the problem.

The conclusions have limits. All model selection used one validation split with only 218 negative reviews, so small differences should not be over-interpreted. The sample-level analysis in Section 7 examines the initial ensemble, not the final one, and the noise experiment uses synthetic noise and a linear model. Natural next steps are to repeat the error analysis for the final ensemble, to evaluate the models on real hotel reviews, and to test noise-robust losses with RoBERTa.

A Code and Reproducibility

All code, data splits and outputs are in the submitted archive. The main files are:

- `README.md`: installation and the commands that reproduce every result;
- `src/common.py`: data loading, the shared validation split, metrics, threshold selection and the writer of `submission.csv`;
- `src/tfidf_models.py`, `src/tfidf_dim_study.py` and `src/resource_table.py`: the TF-IDF models, the dimensionality study and the resource table (Sections 5 and 8);
- `src/noise_exp.py`: the label-noise experiment (Section 10);
- `src/glove_bilstm.py`: GloVe features, mean-embedding classifier and BiLSTM;
- `src/transformer_ft_warmup.py`, `src/transformer_ft.py` and `src/ensemble.py`: RoBERTa fine-tuning and the final predictions;
- `splits/val_idx.npy`: the shared validation indices;
- `outputs/<model>/`: validation and test probabilities, metrics and hyper-parameter logs (for example `outputs/bilstm/hparam_log.csv`); `outputs/noise_exp/summary.csv` holds the full noise-experiment results.

Random seeds are fixed (42, 43 and 44 for the neural models), so the saved outputs can be regenerated.

References

[1] Mayur Wankhade, Annavarapu Chandra Sekhara Rao, and Chaitanya Kulkarni. A survey on sentiment analysis methods, applications, and challenges. *Artificial Intelligence Review*, 55(7):5731–5780, 2022.

[2] Wenxuan Zhang, Xin Li, Yang Deng, Lidong Bing, and Wai Lam. A survey on aspect-based sentiment

analysis: Tasks, methods, and challenges. *IEEE Transactions on Knowledge and Data Engineering*, 35 (11):11019–11038, 2023.

[3] John Blitzer, Mark Dredze, and Fernando Pereira. Biographies, Bollywood, boom-boxes and blenders: Domain adaptation for sentiment classification. In *Proceedings of the 45th Annual Meeting of the Association of Computational Linguistics (ACL)*, pages 440–447, 2007.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019.

[5] Wenxuan Zhang, Yue Deng, Bing Liu, Sinno Pan, and Lidong Bing. Sentiment analysis in the era of large language models: A reality check. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 3881–3906, 2024.

[6] Constantinos Karouzos, Georgios Paraskevopoulos, and Alexandros Potamianos. UDALM: Unsupervised domain adaptation through language modeling. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2579–2590, 2021.

[7] Qingyu Tan, Ruidan He, Lidong Bing, and Hwee Tou Ng. Domain generalization for text classification with memory-based supervised contrastive learning. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 6916–6926, 2022.

[8] Yun Luo, Fang Guo, Zihan Liu, and Yue Zhang. Mere contrastive learning for cross-domain sentiment analysis. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 7099–7111, 2022.

[9] Orith Toledo-Ronen, Matan Orbach, Yoav Katz, and Noam Slonim. Multi-domain targeted sentiment analysis. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 2751–2762, 2022.

[10] Jin Wang, Liang-Chih Yu, and Xuejie Zhang. SoftMCL: Soft momentum contrastive learning for fine-grained sentiment-aware pre-training. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 15012–15023, 2024.

[11] Yang Zhao, Masayasu Muraoka, Issei Yoshida, Bishwaranjan Bhattacharjee, and Hiroshi Kanayama. A simple-yet-efficient instruction augmentation method for zero-shot sentiment classification. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 1585–1599, 2025.

[12] Dipankar Srirag, Aditya Joshi, Jordan Painter, and Diptesh Kanojia. BESSTIE: A benchmark for sentiment and sarcasm classification for varieties of English. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 8413–8429, 2025.

[13] Erik Cambria, Qian Liu, Sergio Decherchi, Frank Xing, and Kenneth Kwok. SenticNet 7: A commonsense-based neurosymbolic AI framework for explainable sentiment analysis. In *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pages 3829–3839, 2022.

[14] Erik Cambria, Xulang Zhang, Rui Mao, Melvin Chen, and Kenneth Kwok. SenticNet 8: Fusing emotion AI and commonsense AI for interpretable, trustworthy, and explainable affective computing. In *HCI International 2024 – Late Breaking Papers*, pages 197–216, 2024.

[15] Tomas Mikolov, Kai Chen, Greg Corrado, and Jeffrey Dean. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*, 2013.

[16] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543, 2014.

[17] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023.

[18] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021.